

КЛАССИЧЕСКОЕ МАШИННОЕ ОБУЧЕНИЕ · НЕДЕЛЯ 1

Первый ML-эксперимент, которому можно начать доверять

**Как из реальной задачи получить ML-эксперимент,
результату которого можно хотя бы начать доверять?**

Рабочее определение

Машинное обучение — область, изучающая методы, которые позволяют строить и настраивать модели на основе данных или опыта.

область

границы проводят по-разному

методы

строят или настраивают модель

данные или опыт

источник обучающего сигнала

В этом курсе основная рабочая конструкция будет предсказательной: исторические данные → алгоритм обучения → обученная модель → прогноз.

Сначала обучение, затем применение

исторические данные

признаки и известные ответы

→ алгоритм обучения

→ обученная модель

новый объект

только признаки

→ обученная модель

→ прогноз

Правильный ответ нового объекта не передаётся модели при прогнозе.

Ландшафт машинного обучения

Широкая область; наш курс охватывает один важный её фрагмент



1. Парадигмы обучения

- обучение с учителем
- обучение без учителя
- обучение с подкреплением
- самоконтролируемое обучение (self-supervised)



2. Типы задач

- классификация
- регрессия
- ранжирование
- кластеризация
- поиск аномалий
- рекомендации



3. Семейства моделей

- линейные модели
- k-ближайших соседей
- деревья решений
- ансамбли
- ядерные методы
- нейронные сети



4. Области применения

- кредитный скоринг
- антифрод
- прогнозирование оттока
- прогнозирование спроса
- динамическое ценообразование
- медицина
- компьютерное зрение
- NLP и речь
- рекомендательные системы

Инженерный слой



сбор данных



подготовка признаков



обучение



deployment



monitoring



Фокус курса: обучение с учителем → табличные данные → классические ML-модели

Это ориентир, а не строгая таксономия.

Как читать эту карту

Одну систему машинного обучения можно одновременно описать через **режим обучения, тип задачи, семейство моделей и прикладную область**.

Классификацию документов можно описать как обучение с учителем, классификацию и задачу из области NLP.

Решать её можно моделями разных семейств: линейной моделью, деревом, ядерным методом или нейронной сетью.

Фокус курса: обучение с учителем на табличных данных и классические ML-модели.

Чем мы в этом курсе заниматься не будем

- глубокое обучение
- NLP и обработка речи
- reinforcement learning
- полный курс по причинному выводу
- computer vision
- рекомендательные системы и временные ряды
- generative AI, foundation models, LLM, agents
- data engineering, MLOps и промышленная эксплуатация

Современный ML намного шире одной учебной дисциплины. Каждое из этих направлений требует отдельного связного курса.

На чём сосредоточимся

Классическое машинное обучение, прежде всего обучение с учителем на табличных данных.

постановка задачи · функция потерь · оптимизация · обобщающая способность
проверка модели · сложность · интерпретация

В классических моделях эти механизмы видны относительно прозрачно — и затем возвращаются в более сложных системах.

Классический ML в прикладных задачах

Кредитный скоринг

вероятность дефолта

Антифрод

вероятность мошеннической
операции

Отток клиентов

риск отмены подписки

Прогноз спроса

будущий объём продаж или заказов

Оценка стоимости

например, цена недвижимости

Операционные риски

отказ оборудования или просрочка

| объекты + признаки + известный результат → прогноз для новых объектов

Маршрут курса · Недели 1–5

1. Осмысленный ML-эксперимент

постановка · train/test · бейзлайн · недообучение и переобучение

2. Линейная регрессия

МНК · аналитическое решение · матричная запись · остатки

3. Функции потерь и градиентный спуск

MSE и MAE · градиент · learning rate · scaling

4. Бинарная логистическая регрессия

score · sigmoid · вероятность · log-loss · threshold

5. Метрики классификации и решение

precision/recall · F-мера · ROC/PR · выбор порога

Маршрут курса · Недели 6–10

6. Регуляризация и проверка модели

Ridge/Lasso · validation · cross-validation

7. kNN и многоклассовая классификация

соседи · расстояния · softmax · multiclass metrics

8. Обучаемые преобразования, Pipeline и утечки

пропуски · категории · ColumnTransformer · leakage

9. Решающие деревья

разбиения · глубина · критерии остановки

10. Коллоквиум

понимание и применение материала недель 1–9

Маршрут курса · Недели 11–14

11. Bias–variance, bagging и Random Forest

variance · bootstrap · усреднение · случайные признаки

12. Градиентный бустинг

аддитивная модель · псевдоостатки · learning rate · early stopping

13. Современный tabular ML, HPO и диагностика

CatBoost/XGBoost/LightGBM · поиск · анализ ошибок · interpretation

14. Защита проектов

перенос общего ML-подхода в новую область

Из чего складываются 10 баллов

$$\text{Итог} = \text{ДЗ}_1 + \text{ДЗ}_2 + \text{Проект} + \text{Коллоквиум} + \text{Семинары}$$

$$1 + 1 + 3 + 3 + 2 = 10$$

ДЗ 1 / ДЗ 2 — по 1 баллу

Устный коллоквиум — 3 балла

Проект — 3 балла

Активность на семинарах — 2 балла

Итогового экзамена нет.

Домашние работы

ДЗ 1

1 балл

ДЗ 2

1 балл

Темы домашних работ будут объявлены отдельно.

Устный коллоквиум

3 балла

Устно

Неделя 10

Первая часть курса

| Понимание механизмов и рассуждение на новой небольшой задаче.

Проект

3 балла

Разобраться в соседней области ML.

Понять центральный механизм.

Показать небольшой эксперимент.

PCA и k-means

снижение размерности и кластеризация

Нейронные сети

MLP и backpropagation

Временные ряды

прогнозирование и temporal backtesting

Текст

Naive Bayes и TF-IDF + логистическая регрессия

A/B-тестирование

randomized experiment и causal effect

Рекомендательные системы

collaborative filtering и matrix factorization

Подробные постановки, данные и требования будут опубликованы отдельно.

Таким образом

- Современный ML можно описывать по режиму обучения, типу задачи, моделям и области применения.
- Курс сосредоточен на классическом обучении с учителем для табличных данных.
- Маршрут идёт от постановки и линейных моделей к ансамблям и диагностике.
- Оценка складывается из двух домашних работ, проекта, коллоквиума и семинарской активности.

ЧАСТЬ I · ОТ ПРИКЛАДНОГО ВОПРОСА К МАТЕМАТИЧЕСКОЙ ЗАДАЧЕ

Кейс: список клиентов для команды удержания

Сервис работает по ежемесячной подписке и имеет десятки тысяч активных аккаунтов.

Команда удержания может связаться только с **200 клиентами в день**.

Каждый вечер, в момент t_0 , требуется оценить риск отмены подписки **в следующие 30 дней** и сформировать список контактов на завтра.

Это учебный синтетический пример

Какой список нужно отдать команде завтра?

Как выглядит историческая строка

prediction time = 31.01.2026 23:59

customer_id	tenure_m	act_d_14d	late_p_90d	supp_t_30d	churn_30d
1042	5	3	1	2	1
2718	28	12	0	0	0
3195	11	7	0	1	0

- tenure_m

— месяцев в сервисе
- late_p_90d

— просроченных платежей за 90 дней
- churn_30d

— отмена в следующие 30 дней
- act_d_14d

— активных дней за 14 дней
- supp_t_30d

— обращений за 30 дней

Объект, признаки, цель и горизонт

Объект

активный аккаунт в момент t_0

Признаки x

сведения об аккаунте, доступные в t_0

Целевая переменная y

`churn_30d` : отмена в следующие 30 дней

Горизонт

интервал будущего длиной 30 дней

Тот же клиент через месяц — другое наблюдение: состояние аккаунта изменилось.

Модель как правило прогноза

Модель переводит признаки объекта в прогноз.

$$f : \mathcal{X} \longrightarrow \mathcal{Y}, \quad \hat{y} = f(x)$$

$\mathcal{X}$

$\mathcal{Y} = \{0, 1\}$

пространство признаков описаний

возможные ответы churn-задачи

Семейство и обученная модель

 $\mathcal{F}$ $\hat{f} \in \mathcal{F}$

семейство моделей

обученная модель

Какие правила допустимы до обучения?

Какое конкретное правило получено после обучения?

Возможные семейства: линейные правила, деревья, базовые постоянные прогнозы и другие.

Что называется обучением

Обучение — процесс, в котором по историческим данным выбирается или настраивается модель для последующего применения к новым объектам.

$$D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$$

$$\hat{f} = A(D_{\text{train}}), \quad \hat{y} = \hat{f}(x)$$

D_{train} — исторические примеры для обучения

A — алгоритм обучения

$\hat{f}$ — модель, полученная после обучения

$\hat{y}$ — прогноз модели для нового объекта

Более подробная параметрическая запись приведена в приложении.

Проще говоря: что происходит при обучении

1 · Данные

Исторические примеры, для которых известны признаки и правильные ответы.

2 · Алгоритм обучения

Использует примеры, чтобы выбрать или настроить правило.

3 · Обученная модель

Хранит полученное правило и применяется к новым объектам.

данные → алгоритм обучения → обученная модель → прогноз для нового объекта

Момент прогноза проводит границу информации

Какая информация существует в момент прогноза?



Утечка из будущего

Утечка будущего (future leakage)

В признаке есть информация, недоступная в момент t_0 .

Утечка цели (target leakage)

Признак прямо или косвенно сообщает будущий ответ.

Было ли значение доступно именно тогда, когда требовался прогноз?

Тот же принцип работает в кредитном скоринге, антифроде или медицинских прогнозах: признак должен быть известен в момент, когда требуется прогноз.

Три вопроса к одному churn-кейсу

Вопрос	Желаемый результат	Возможные методы
Кто уйдёт?	прогноз для нового клиента	предсказательная модель + оценка на отложенных данных
Какие характеристики связаны с уходом и насколько уверенно?	связь и её неопределённость	статистическая модель + оценка неопределённости
Кому звонок поможет остаться?	причинный эффект контакта	рандомизированный эксперимент / причинные методы

Высокий риск ухода не означает высокий эффект звонка.

Таким образом

- Объект, признаки, цель, горизонт и t_0 задают churn-задачу.
- Алгоритм обучения использует исторические данные и возвращает конкретную модель.
- Признак допустим, только если существует в момент прогноза.
- Прогноз, статистический вывод и причинный эффект требуют разных методов и дизайна исследования.

Пока мы ещё ничего не сказали о качестве модели

ЧАСТЬ II · ПРОВЕРКА ПРОГНОЗА НА НОВЫХ ОБЪЕКТАХ

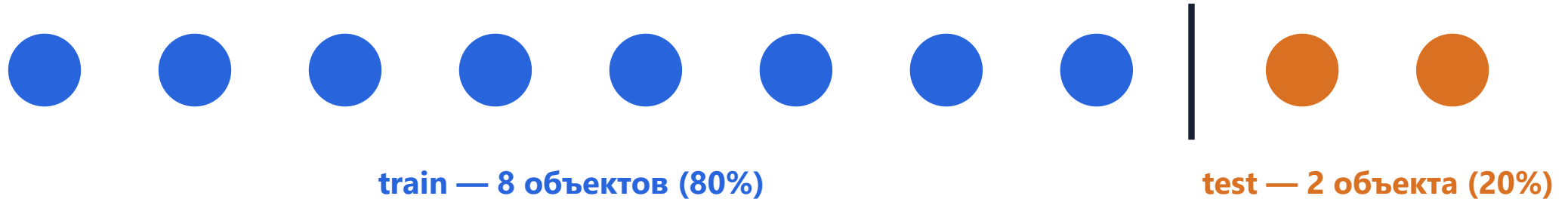
Что говорит качество модели на тех же данных, по которым она обучалась?

Обобщающая способность — способность модели сохранять качество на новых объектах.

Простое разделение: 8 + 2

Модель обучаем на **train**. Качество прогноза оцениваем на данных, которые в обучении не участвовали.

Часть исторических объектов временно играет роль новых



80/20 — только наглядный пример, универсальной доли нет

Train и test отвечают на разные вопросы

train

Используется для построения и настройки модели.

Ошибка описывает знакомые объекты.

test

Используется как **независимая проверка** качества на новых объектах, если разбиение разумно имитирует будущее применение модели.

Обучающая и тестовая выборки играют разные роли в процессе обучения

Как исследователь подстраивается под test

1. Обучили вариант A → посмотрели качество на test.
2. Изменили признаки → снова посмотрели тот же test.
3. Повторили цикл 20 раз → оставили лучший вариант.

Test повлиял на решения исследователя. Полученная оценка больше не является полностью независимой.

Утечка будущего и повторная обратная связь по test нарушают одно правило: при разработке используется информация, которая должна оставаться недоступной.

Как тогда сравнивать модели?

Финальный test стараются оставить независимым.

Для итеративного выбора используют отдельную валидационную выборку (validation).

Бейзлайн задаёт точку отсчёта

95% точности

Это хороший результат?

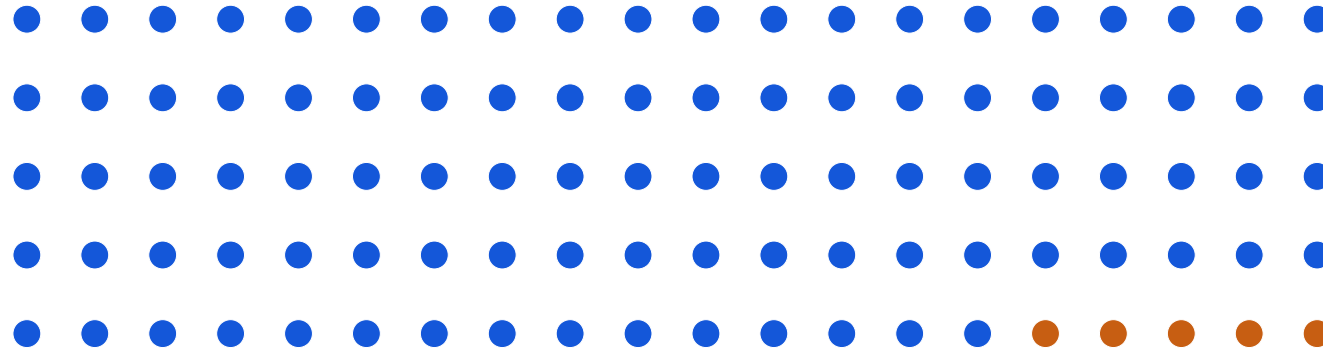
Значение метрики само по себе почти ничего не говорит, пока не понятно, с чем его сравнивать.

Бейзлайн — простой понятный прогноз или правило, задающее масштаб.

95 правильных ответов из 100

95 объектов класса 0

5 объектов класса 1



≈ 50%

равновероятное угадывание

95%

всегда прогнозировать 0

следующий шаг

предметное правило → обучаемая модель

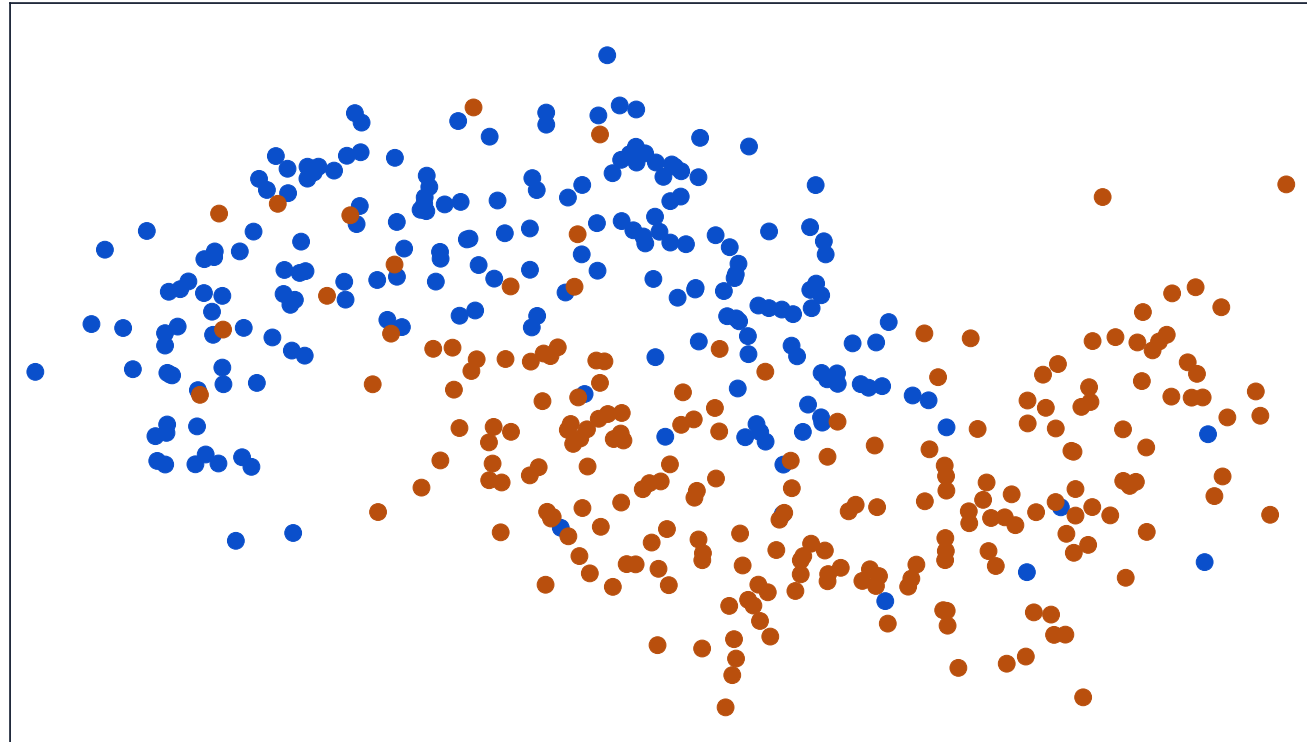
Правило «всегда 0» не обнаружит ни одного редкого объекта класса 1.

Таким образом

- Обобщающая способность относится к качеству на новых объектах.
- На train обучаем модель, на test независимо проверяем её
- Повторная обратная связь по test разрушает независимость оценки.
- Бейзлайн задаёт точку отсчёта для интерпретации метрики.

ЧАСТЬ III · НЕДООБУЧЕНИЕ И ПЕРЕОБУЧЕНИЕ

Сначала только данные

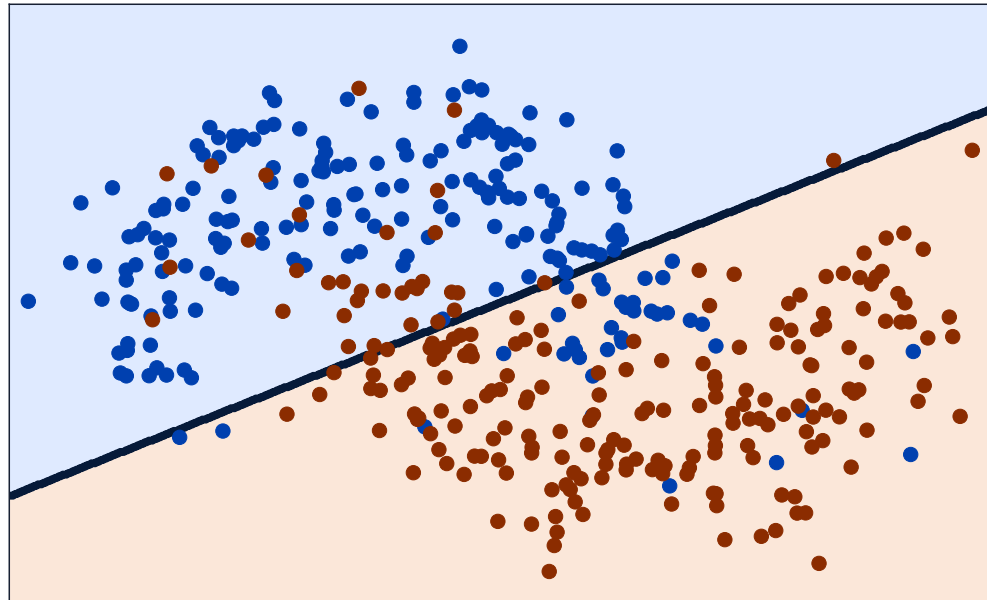


Устойчивая структура: две изогнутые области.

Шум: отдельные нетипичные точки.

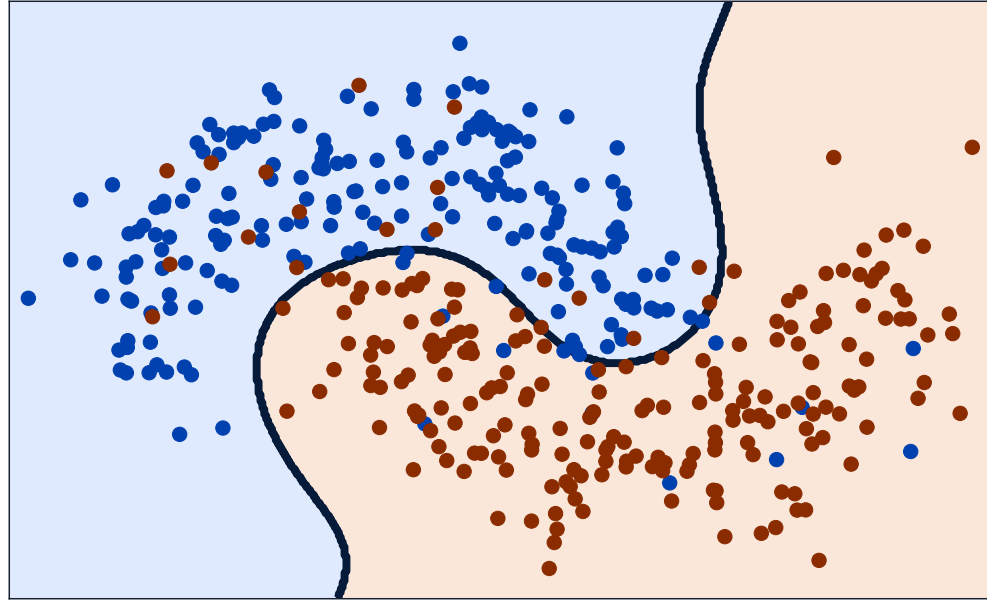
Недообучение

Недообучение: модель или процедура обучения недостаточно гибки, чтобы описать устойчивую закономерность в данных.



Ошибка высока и на train, и на новых данных.

Разумная гибкость

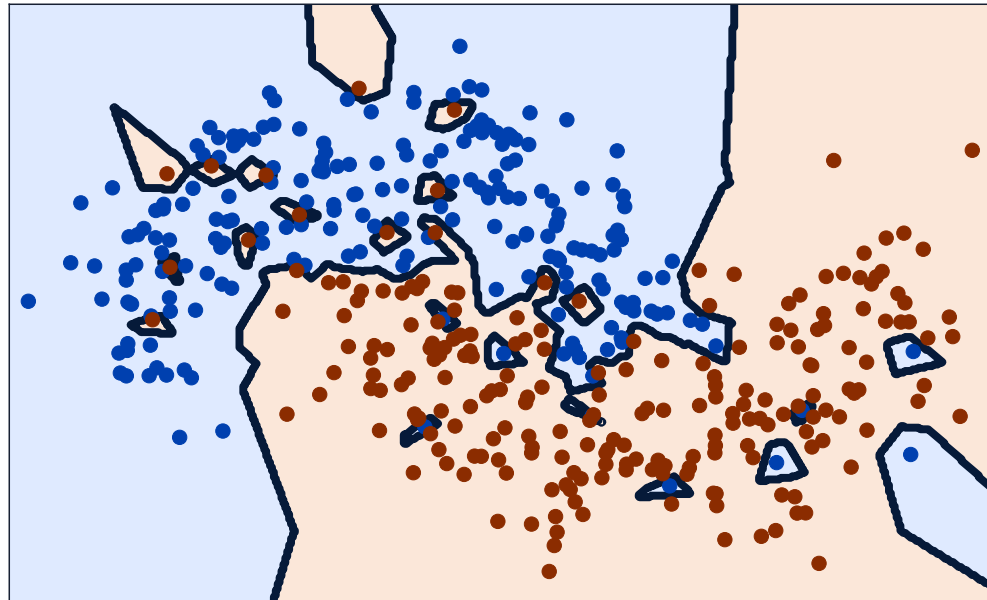


Граница описывает форму двух областей.

Нетипичная точка не получает собственный островок.

Переобучение

Переобучение: правило чрезмерно чувствительно к конкретной обучающей выборке и повторяет случайные детали.

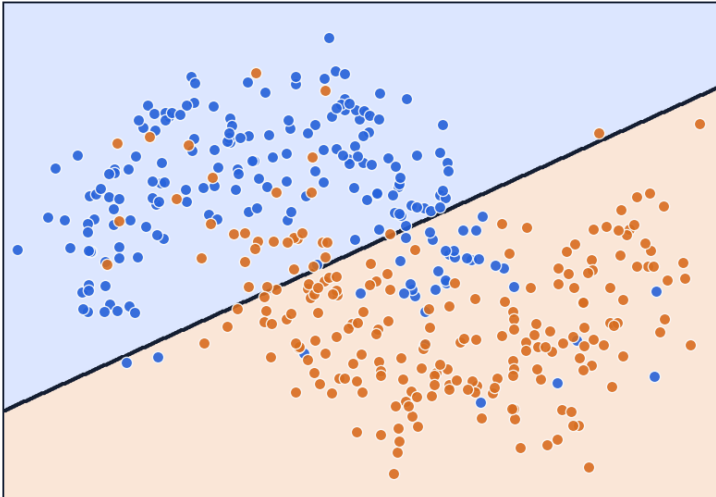


Ошибка на train очень мала, а на новых данных растёт.

Одна выборка, три уровня гибкости

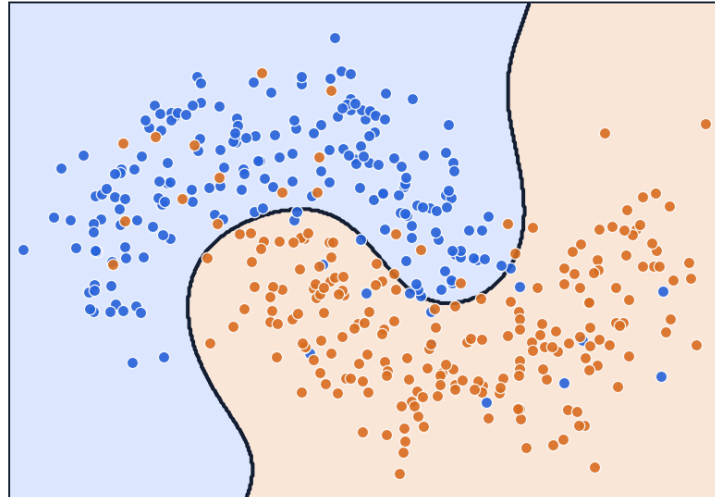
Гибкость сначала помогает описать структуру, затем начинает повторять шум

Недообучение



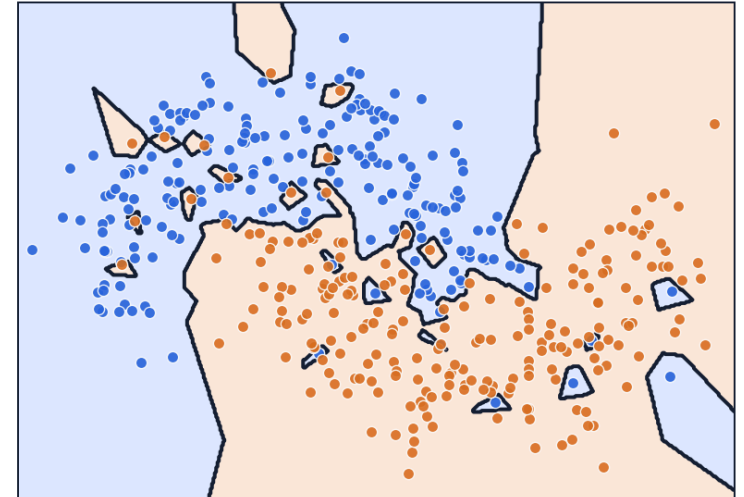
гладкая граница пропускает устойчивую форму

Разумная гибкость



граница следует двум основным областям

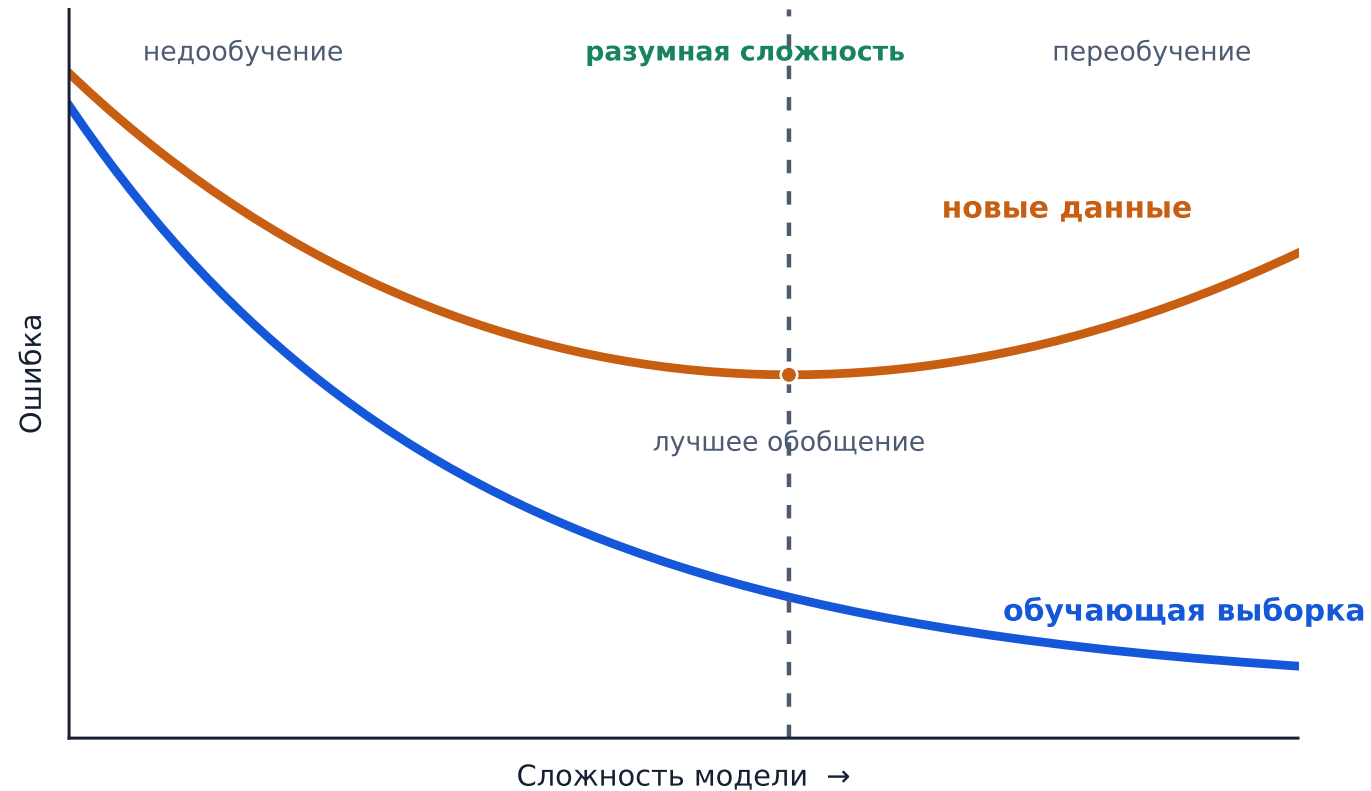
Переобучение



локальные островки повторяют отдельные метки

Сложность модели и две ошибки

Частая картина при увеличении гибкости модели



Ось сложности порядковая и зависит от семейства моделей.

Проще говоря

Слишком просто

Модель не умеет описать часть устойчивой закономерности.

Разумно гибко

Модель улавливает основную структуру данных.

Слишком гибко

Появляются способы подстроиться под случайные детали этой выборки.

Очень маленькая ошибка на train сама по себе ещё мало говорит о качестве будущих прогнозов.

Что говорит разрыв между ошибками

Большой разрыв

Типичный признак переобучения: на train хорошо, на новых данных заметно хуже.

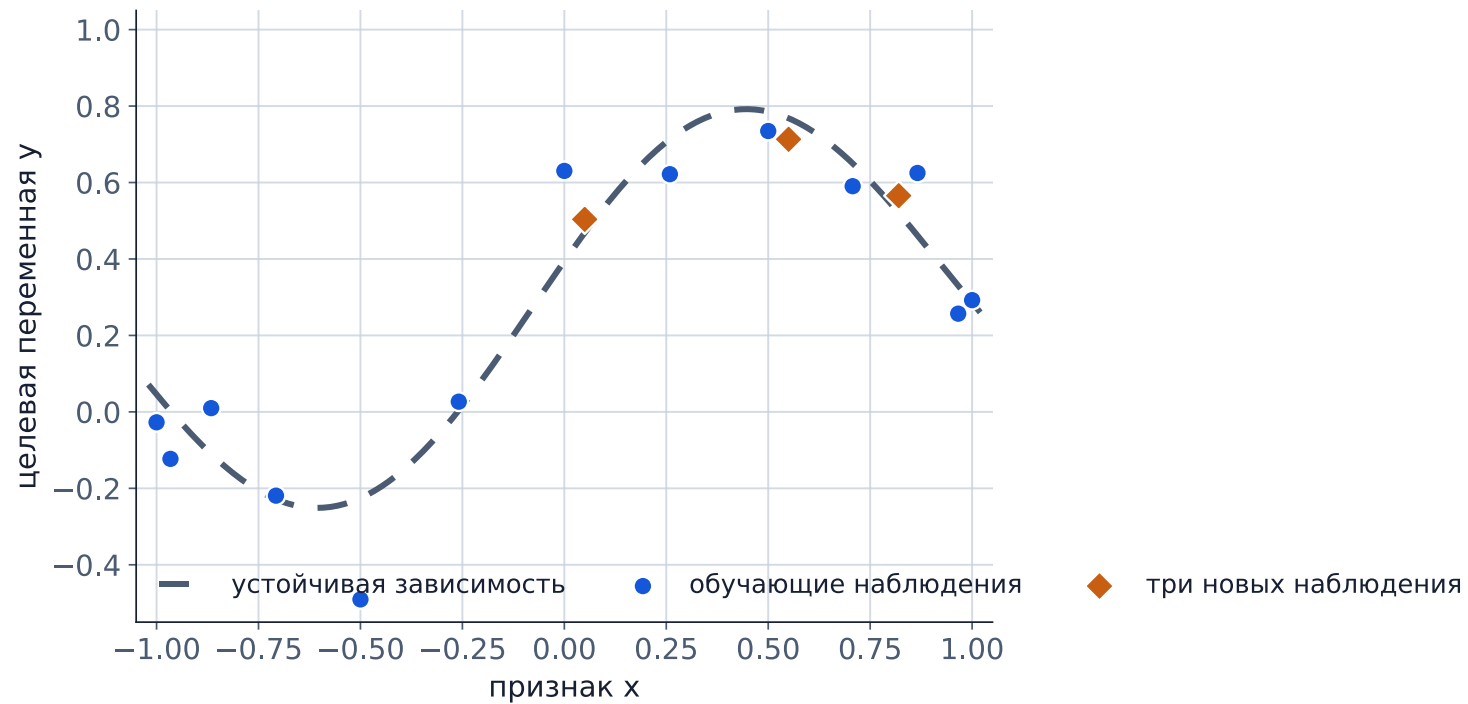
Маленький разрыв

Совместим и с хорошей моделью, и с одинаково плохим качеством на обеих выборках.

Оцениваем и величину разрыва, и само качество модели — в том числе по сравнению с бейзлайном.

Регрессия: процесс порождения данных

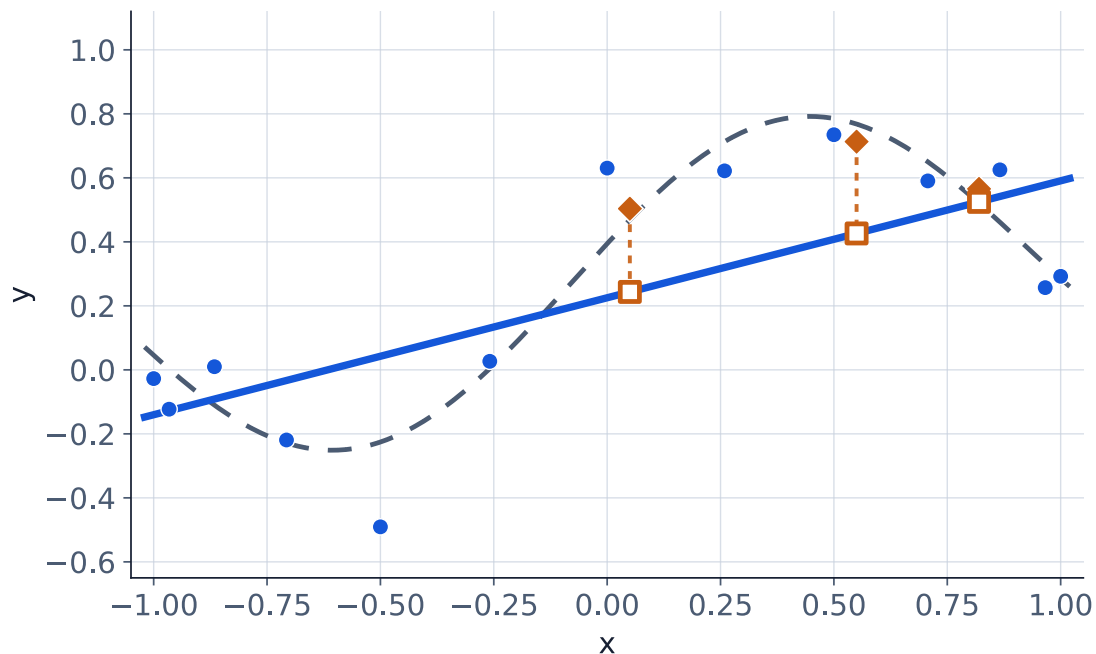
$$g(x) = 0.28 + 0.46 \sin(\pi(x + 0.08)) + 0.12x, \quad y_i = g(x_i) + \varepsilon_i$$



Синие точки используются для обучения. Оранжевые ответы модель при обучении не видит.

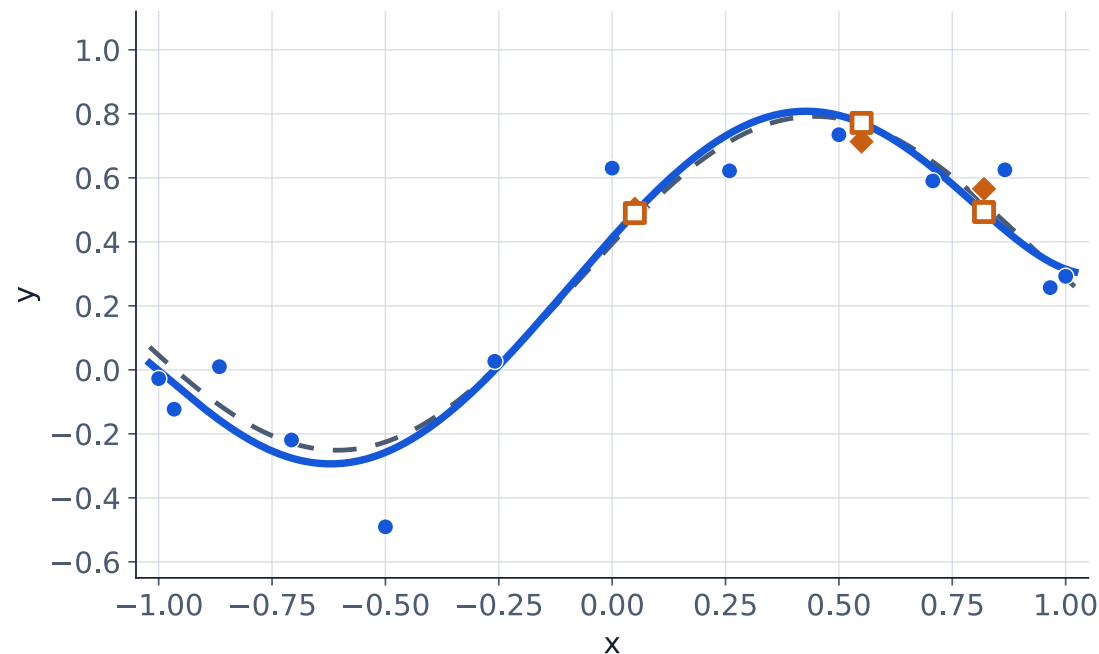
Недостаточная и разумная гибкость

Степень 1



Степень 1: устойчиво пропускает нелинейный изгиб.

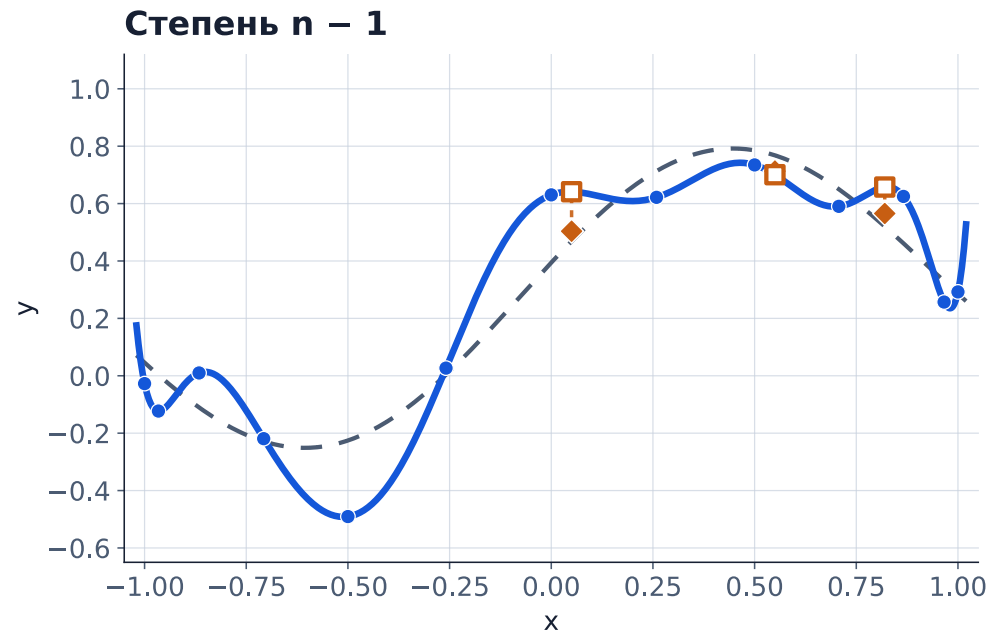
Умеренная степень



Умеренная степень: передаёт основную форму между наблюдениями.

Интерполяция: ноль на train

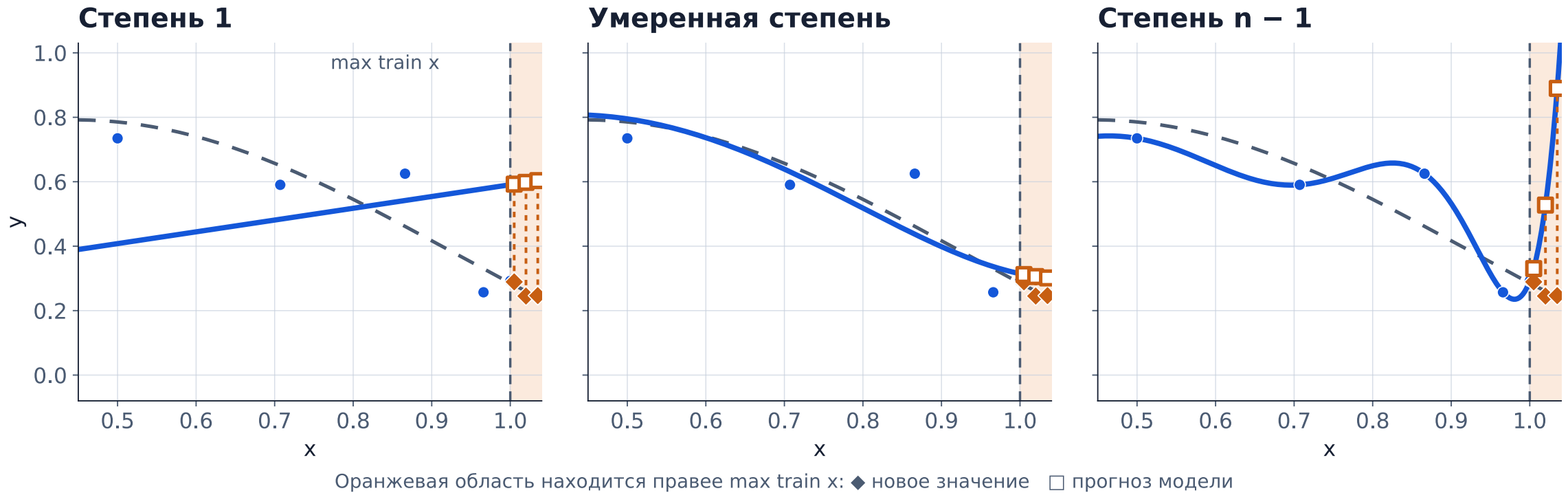
Для n точек с попарно различными x_i существует единственный полином степени не выше $n - 1$, проходящий через все эти точки.



Нулевая train error не гарантирует хороший прогноз на новых наблюдениях.

А что будет за пределами данных?

Экстраполяция — прогноз для значений признака за пределами диапазона обучающих данных.



Линейная модель даёт простое и предсказуемое по форме продолжение, но оно тоже может быть неверным.

Таким образом

- **Недообучение:** высокая ошибка на train и новых объектах.
- Разумная гибкость описывает устойчивую структуру.
- **Переобучение:** ошибка на train продолжает падать, ошибка на новых данных растёт.
- Экстраполяция относится к прогнозу уже за границей обучающего диапазона.

ЧАСТЬ IV · НЕМНОГО ООП: МОДЕЛЬ КАК ОБЪЕКТ PYTHON

Знакомые значения в Python — тоже объекты

```
balance = 1500
customer_id = "C-1042"
active_days = [1, 4, 8]

print(type(balance))      # <class 'int'>
print(type(customer_id))  # <class 'str'>
print(type(active_days))  # <class 'list'>
```

Мы уже работали с объектами и классами Python, даже если не использовали эту терминологию. Числа, строки и списки — объекты соответствующих классов

Класс, экземпляр, атрибут и метод

Класс

описание типа объектов: данных и поддерживаемых операций

пример: `list`

Атрибут

именованное значение, связанное с объектом

составляет состояние

Экземпляр

конкретный объект данного класса

пример: `active_days`

Метод

функция класса, вызываемая для экземпляра

пример: `append`

Один компактный класс: Account

```
class Account:
    def __init__(self, customer_id):
        self.customer_id = customer_id
        self.active = True

    def cancel(self):
        self.active = False
```

`__init__` автоматически вызывается при создании экземпляра и задаёт его начальное состояние.

Класс

Account

Атрибуты

customer_id, active

Метод

cancel()

Создаём экземпляр и вызываем метод

```
account = Account(customer_id="C-1042")

print(type(account))    # <class '__main__.Account'>
print(account.active)   # True

account.cancel()
print(account.active)   # False
```

До `cancel()`

```
account.active == True
```

После `cancel()`

```
account.active == False
```

`__main__` означает, что класс определён непосредственно в исполняемом сейчас скрипте или notebook.

Что означает `self`

```
def cancel(self):  
    self.active = False
```

`self` — ссылка на экземпляр, для которого сейчас выполняется метод.

При вызове `account.cancel()` имя `self` внутри метода указывает на объект `account`. Поэтому меняется его атрибут `active`.

Обычный объект → объект модели

Модель scikit-learn — экземпляр класса со своим состоянием и методами.

```
fit(X, y)
```

Использует обучающие данные и изменяет состояние экземпляра.

```
predict(X_new)
```

Читает сохранённое состояние и выдаёт ответы для новых строк.

`predict` **использует уже обученную модель.**

Разработаем простой классификатор

Идея: во время обучения запомнить самый частый класс, а затем предсказывать его для каждого нового объекта.

1. Посмотреть на ответы `y` в обучающей выборке.
2. Определить и сохранить самый частый класс.
3. Вернуть сохранённый класс для каждой новой строки.

Полный класс: смотрим на `fit`

```
class MostFrequentClassifier:
    def fit(self, X, y):
        ones = sum(y)
        zeros = len(y) - ones
        self.class_ = 1 if ones > zeros else 0
        return self

    def predict(self, X):
        return [self.class_] * len(X)
```

`ones`, `zeros` — локальные переменные метода.

`self.class_` сохраняется в состоянии модели после `fit`.

В scikit-learn суффикс `_` отмечает публичные атрибуты, появившиеся во время `fit`: `class_`, `coef_`, `classes_`.

Тот же класс: смотрим на `predict`

```
class MostFrequentClassifier:
    def fit(self, X, y):
        ones = sum(y)
        zeros = len(y) - ones
        self.class_ = 1 if ones > zeros else 0
        return self

    def predict(self, X):
        return [self.class_] * len(X)
```

`self.class_` — читаем состояние, сохранённое после обучения.

`len(X)` — выдаём один прогноз на каждую строку.

Данные маленького эксперимента

train

```
X_train = [  
    [5, 3],  
    [28, 12],  
    [11, 7],  
    [2, 1],  
    [19, 10],  
]  
y_train = [1, 0, 0, 1, 0]
```

test

```
X_test = [  
    [3, 1],  
    [14, 8],  
    [7, 4],  
]  
y_test = [1, 0, 0]
```

X : каждая строка — `[tenure_m, act_d_14d]` . y : отменил ли клиент подписку в следующие 30 дней.

Что изменилось после `fit`?

до fit

```
model = MostFrequentClassifier()  
vars(model)  
# {}
```

после fit

```
model.fit(X_train, y_train)  
vars(model)  
# {'class_': 0}
```

`vars(model)` позволяет посмотреть атрибуты, которые сейчас хранит экземпляр.

Прогноз и ручной расчёт accuracy

```
y_pred = model.predict(X_test)
# [0, 0, 0]
```

Тестовый объект	y_test	y_pred	Совпадение
1	1	0	нет
2	0	0	да
3	0	0	да

$$\text{accuracy} = \frac{2}{3} \approx 0.667$$

Это оценка бейзлайна самого частого класса на трёх тестовых объектах; для вывода о практической полезности churn-системы её недостаточно.

Мост к DummyClassifier

```
from sklearn.dummy import DummyClassifier

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
y_pred = baseline.predict(X_test)
# [0 0 0]
```

Библиотечный класс поддерживает больше возможностей, однако основная последовательность та же: **создание объекта** → `fit` → `predict`.

Таким образом

- Класс описывает устройство объектов, а экземпляр хранит собственное состояние.
- `__init__` задаёт начальное состояние; методы обращаются к экземпляру через `self`.
- `fit` добавляет состояние, полученное по обучающим данным; `predict` использует его для новых строк.
- `MostFrequentClassifier` и `DummyClassifier` реализуют один бейзлайн через общий интерфейс.

Первый проверяемый ML-эксперимент целиком



ИТОГ НЕДЕЛИ 1

Что мы теперь называем ML-экспериментом

- ML-задача начинается с определения объектов, признаков, целевой переменной и момента прогноза.
- Алгоритм обучения использует исторические данные и возвращает модель, которую применяют к новым объектам.
- Качество прогноза проверяют на данных, не участвовавших в обучении, и сравнивают с понятным бейзлайном.
- Низкая ошибка на обучающей выборке ещё не означает хорошего обобщения: возможны недообучение и переобучение.

Следующая неделя: линейная регрессия — первая модель, механизм которой разберём подробно.

A01 · Параметрическая запись

$$\mathcal{F} = \{f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y} \mid \theta \in \Theta\}$$

$$\hat{\theta} = A(D_{\text{train}}), \quad \hat{y} = f_{\hat{\theta}}(x)$$

Запись с θ — удобная общая схема. Она не требует, чтобы у любой ML-модели буквально существовал конечномерный вектор параметров.

A02 · Атрибут instance и атрибут класса

```
class Account:
    category = "subscription"

    def __init__(self, customer_id):
        self.customer_id = customer_id
```

`self.customer_id`

принадлежит конкретному экземпляру

`Account.category`

задан в теле класса и относится к классу

Для основной логики Недели 1 это различие не требуется.

A03 · Что почитать после лекции

James G., Witten D., Hastie T., Tibshirani R., Taylor J. — *An Introduction to Statistical Learning: with Applications in Python*, §§2.1–2.2.1.
doi.org/10.1007/978-3-031-38747-0

Полезно для связи между прогнозом, ошибкой на train/test, гибкостью модели и переобучением.

Соколов Е. А. — *Машинное обучение — 1. Лекция 1: Введение в машинное обучение*.

github.com/esokolov/ml-course-hse/.../lecture01-intro.tex

Более формальная постановка ML-задачи через пространства объектов и ответов, семейство моделей, функцию ошибки и алгоритм обучения.

Wilber J., Werness B. — *The Bias–Variance Tradeoff*. MLU-Explain.

mlu-explain.github.io/bias-variance/

Интерактивная интуиция недообучения, переобучения и чувствительности результата к конкретной выборке.